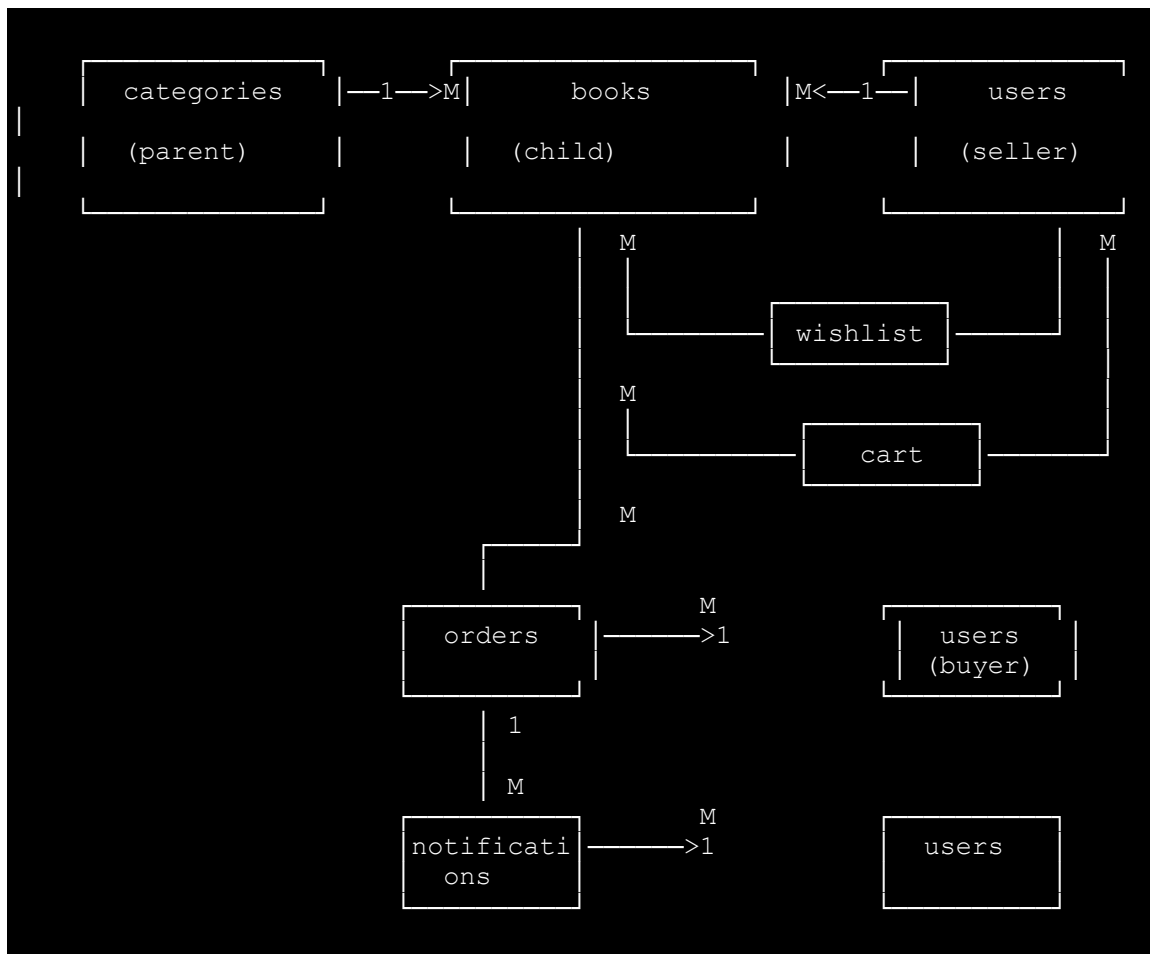


Dokumentasi Relasi Database — EbookStore

Diagram Relasi Antar Tabel



1. Tabel **categories** → **books**

Relasi: One-to-Many (1 : M)

Key Type	Kolom Parent	Kolom Child
Primary Key	<code>categories.id</code>	—
Foreign Key	<code>categories.nama</code>	<code>books.kategori</code>

Constraint: FOREIGN KEY (`kategori`) REFERENCES `categories(nama)` ON DELETE RESTRICT ON UPDATE CASCADE

Penjelasan:

- Satu kategori (misalnya "Fiksi") dapat digunakan oleh **banyak buku**.
 - Sebaliknya, **satu buku hanya memiliki tepat satu kategori** (kolom `kategori` di tabel `books` adalah string tunggal, bukan array).
 - **ON DELETE RESTRICT**: Kategori tidak dapat dihapus jika masih ada buku yang menggunakan kategori tersebut. Ini menjaga integritas data agar tidak ada buku dengan kategori yang sudah tidak ada.
 - **ON UPDATE CASCADE**: Jika nama kategori diubah, semua buku yang merujuk pada kategori tersebut akan otomatis mengikuti perubahan. Contoh: "Fiksi" diubah menjadi "Fiksi Modern" → semua buku dengan kategori "Fiksi" otomatis berubah menjadi "Fiksi Modern".
-

2. Tabel `users` (sebagai seller) → `books`

Relasi: One-to-Many (1 : M)

Key Type	Kolom Parent	Kolom Child
Primary Key	<code>users.id</code>	—
Foreign Key	—	<code>books.seller_id</code>

Constraint: `FOREIGN KEY (seller_id) REFERENCES users(id) ON DELETE SET NULL ON UPDATE CASCADE`

Penjelasan:

- Satu seller (pengguna dengan role '`seller`') dapat menambahkan **banyak buku** ke toko.
 - Sebaliknya, **satu buku hanya dimiliki oleh tepat satu seller**.
 - **ON DELETE SET NULL**: Jika akun seller dihapus, kolom `seller_id` pada buku-buku yang bersangkutan akan diisi `NULL`, bukan ikut terhapus. Ini mencegah kehilangan data buku secara permanen ketika seller meninggalkan platform.
 - **ON UPDATE CASCADE**: Jika `id` user berubah, nilai `seller_id` di tabel `books` akan ikut berubah.
-

3. Tabel `users` (sebagai buyer) → `orders`

Relasi: One-to-Many (1 : M)

Key Type	Kolom Parent	Kolom Child
Primary Key	<code>users.id</code>	—
Foreign Key	—	<code>orders.buyer_id</code>

Constraint: `FOREIGN KEY (buyer_id) REFERENCES users(id) ON DELETE RESTRICT ON UPDATE CASCADE`

Penjelasan:

- Satu buyer (pembeli) dapat membuat **banyak pesanan**.
 - Sebaliknya, **satu pesanan hanya milik satu buyer**.
 - **ON DELETE RESTRICT:** User pembeli tidak dapat dihapus jika masih memiliki riwayat pesanan. Ini penting untuk kepentingan audit, pelacakan transaksi, dan akuntansi.
 - **ON UPDATE CASCADE:** Jika `id` user berubah, nilai `buyer_id` pada pesanan akan ikut berubah.
-

4. Tabel `books` → `orders`

Relasi: One-to-Many (1 : M)

Key Type	Kolom Parent	Kolom Child
Primary Key	<code>books.id</code>	—
Foreign Key	—	<code>orders.book_id</code>

Constraint: `FOREIGN KEY (book_id) REFERENCES books(id) ON DELETE RESTRICT ON UPDATE CASCADE`

Penjelasan:

- Satu buku dapat dipesan **berkali-kali** dalam pesanan yang berbeda.
- Sebaliknya, **satu baris pesanan hanya memuat satu buku** (setiap order row adalah untuk 1 jenis buku; jika buyer ingin membeli beberapa buku berbeda, akan dibuat beberapa baris di tabel orders).
- **ON DELETE RESTRICT**: Buku tidak dapat dihapus jika sudah memiliki riwayat pemesanan. Ini melindungi data historis dan mencegah order menjadi "yatim piatu" (orphaned) tanpa referensi buku.
- **ON UPDATE CASCADE**: Jika `id` buku berubah, nilai `book_id` pada orders akan mengikuti.

5. Tabel `users` ↔ `books` (melalui `wishlist`)

Relasi: Many-to-Many (M : M) dengan junction table

Junction Table	Kolom FK ke `users`	Kolom FK ke `books`
wishlist	wishlist.user_id	wishlist.book_id

Constraint:

- `FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE ON UPDATE CASCADE`
- `FOREIGN KEY (book_id) REFERENCES books(id) ON DELETE CASCADE ON UPDATE CASCADE`
- `UNIQUE(user_id, book_id)`

Penjelasan:

- Relasi **Many-to-Mary** ini dipecah menggunakan tabel penghubung (junction table) `wishlist` karena SQLite tidak mendukung relasi M:N secara langsung.
- **Satu user dapat menyukai (wishlist) banyak buku** — misalnya user bisa menyimpan 10 buku favorit.
- **Satu buku dapat di-wishlist oleh banyak user** — buku populer bisa di-wishlist oleh puluhan user berbeda.
- **ON DELETE CASCADE**: Jika user dihapus, semua wishlist milik user tersebut ikut terhapus. Jika buku dihapus, semua entry wishlist yang merujuk buku itu juga terhapus. Tidak ada data sampah (orphan).
- **ON UPDATE CASCADE**: Perubahan ID user atau buku akan propagasi ke tabel wishlist.

- **UNIQUE(user_id, book_id):** Mencegah duplikasi — user tidak bisa menambahkan buku yang sama ke wishlist dua kali.
-

6. Tabel `users` ↔ `books` (melalui `cart`)

Relasi: Many-to-Many (M : M) dengan junction table (*rich*)

Junction Table	Kolom FK ke `users`	Kolom FK ke `books`	Atribut Tambahan
<code>cart</code>	<code>cart.user_id</code>	<code>cart.book_id</code>	<code>qty</code> (kuantitas)

Constraint:

- `FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE ON UPDATE CASCADE`
- `FOREIGN KEY (book_id) REFERENCES books(id) ON DELETE CASCADE ON UPDATE CASCADE`
- `UNIQUE(user_id, book_id)`

Penjelasan:

- Sama seperti wishlist, `cart` adalah tabel junction untuk relasi **Many-to-Many** antara user dan buku, tetapi dengan informasi tambahan **kuantitas** (`qty`) — disebut *rich many-to-many relationship*.
 - **Satu user dapat memiliki banyak item di keranjang** — misalnya user memasukkan 3 buku berbeda ke `cart`.
 - **Satu buku bisa ada di keranjang banyak user** — buku yang sama bisa ada di keranjang si A dan si B secara bersamaan.
 - **Kolom `qty` membedakan cart dari wishlist** — user bisa membeli buku yang sama dalam jumlah lebih dari 1 (misalnya 2 eksemplar).
 - **UNIQUE(user_id, book_id):** Satu buku hanya muncul sekali per user dalam `cart`; jika user menambahkan buku yang sudah ada, akan terjadi kenaikan `qty` (bukan baris baru).
 - **ON DELETE CASCADE:** Jika user atau buku dihapus, entry `cart` terkait ikut terhapus.
-

7. Tabel `users` → `notifications`

Relasi: One-to-Many (1 : M)

Key Type	Kolom Parent	Kolom Child
Primary Key	<code>users.id</code>	—
Foreign Key	—	<code>notifications.user_id</code>

Constraint: `FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE ON UPDATE CASCADE`

Penjelasan:

- Satu user dapat menerima **banyak notifikasi** (konfirmasi pesanan, pembayaran diterima, dll).
 - Sebaliknya, **satu notifikasi hanya ditujukan untuk satu user**.
 - **ON DELETE CASCADE:** Jika user dihapus, semua notifikasi user tersebut ikut terhapus — notifikasi tidak relevan jika usernya sudah tidak ada.
 - **ON UPDATE CASCADE:** Perubahan ID user akan propagasi ke tabel notifikasi.
-

8. Tabel `orders` → `notifications` (via `related_order_id`)

Relasi: One-to-Many (1 : M) — logical reference saja

Key Type	Kolom Parent	Kolom Child
Primary Key	<code>orders.id</code>	—
Logical FK	—	<code>notifications.related_order_id</code>

Catatan: Tidak ada constraint `FOREIGN KEY` untuk `related_order_id` — ini murni referensi logis (opsional).

Penjelasan:

- Satu pesanan dapat memiliki **beberapa notifikasi terkait** (misalnya notifikasi "Pesanan Dikonfirmasi", "Pembayaran Diterima", "Pesanan Dikirim").
 - Sebaliknya, **satu notifikasi hanya merujuk ke satu pesanan** (atau tidak sama sekali jika NULL).
 - Tidak ada foreign key constraint karena `related_order_id` bersifat opsional — notifikasi sistem (seperti "Selamat datang") tidak perlu merujuk ke pesanan manapun.
-

Tabel Independen

Tabel `categories`

Tabel `categories` adalah **tabel master/referensi** yang berdiri sendiri. Tidak ada foreign key yang merujuk ke tabel lain. Tabel ini menjadi *parent* bagi tabel `books` melalui kolom `kategori`.

Tabel `users`

Tabel `users` adalah **tabel master pengguna** yang berdiri sendiri. Tabel ini menjadi *parent* bagi tabel `books` (seller), `orders` (buyer), `wishlist`, `cart`, dan `notifications`.

Ringkasan Semua Relasi

#	Parent → Child	Tipe Relasi	ON DELETE	ON UPDATE
1	<code>categories</code> → <code>books</code>	1:M	RESTRICT	CASCADE
2	<code>users</code> (seller) → <code>books</code>	1:M	SET NULL	CASCADE
3	<code>users</code> (buyer) → <code>orders</code>	1:M	RESTRICT	CASCADE
4	<code>books</code> → <code>orders</code>	1:M	RESTRICT	CASCADE
5	<code>users</code> ↔ <code>books</code> (via <code>wishlist</code>)	M:M (junction)	CASCADE	CASCADE

#	Parent → Child	Tipe Relasi	ON DELETE	ON UPDATE
6	users ↔ books (via cart)	M:M (junction + qty)	CASCADE	CASCADE
7	users → notifications	1:M	CASCADE	CASCADE
8	orders → notifications (logical)	1:M	—	—

Glossary Relasi

Istilah	Arti
One-to-Many (1:M)	Satu record di tabel A dapat memiliki banyak record terkait di tabel B, tetapi satu record di tabel B hanya terkait dengan satu record di tabel A.
Many-to-Many (M:M)	Satu record di tabel A dapat memiliki banyak record terkait di tabel B, dan sebaliknya. Diimplementasikan dengan junction table.
Junction Table	Tabel perantara yang menyimpan foreign key dari kedua tabel yang direlasikan secara many-to-many.
ON DELETE RESTRICT	Mencegah penghapusan data parent jika masih ada data child yang merujuk padanya.
ON DELETE SET NULL	Saat parent dihapus, foreign key pada child diisi NULL (data child tetap ada).
ON DELETE CASCADE	Saat parent dihapus, semua data child yang merujuk padanya ikut terhapus.
ON UPDATE CASCADE	Perubahan primary key parent akan otomatis diikuti oleh foreign key child.